

Agentic AI · Week 3 — follow-along sheet

Friday Oct 2, 2:30–4:30 · everything here lives at jjehou-lab.github.io/nrt_agent_week2

GETTING READY · 2:30

1. Download three packages

From the course site:

```
survey-study.zip      (first activity)
Demo1_tissue_analyzer.zip
Demo2_MCP_Tissue_Analyzer.zip
```

Unzip all three.

2. Open claude.ai in a browser

Sign in. No app install needed.

Your account menu should show the NRT organization — that is your Team seat. If it does not, raise a hand; you can still do everything below.

3. First activity — clean the survey

Open notes.md and read it first. **That is the spec** — it says what the data is supposed to look like, and without it “clean this file” means nothing.

Then drag these four files into a new chat:

```
notes.md
data/raw/wave1.csv
data/raw/wave2.csv
scripts/summarize.py
```

... and ask, exactly:

```
Clean the 2024 survey file and document what you
changed.
```

Before you send it: 48 rows, 6 columns. How many problems do you think are in there?

Your guess: _____

What it found: _____

4. Follow along — the same activity, two more ways

- The instructor runs that same prompt in the **Claude desktop app** and in **Claude Code**, on the same folder. Same question, three amounts of reach.
- **Watch the agent loop** — Field Guide slide 9, whose example goal is the prompt you just ran: goal → gathers context → **asks permission** → acts → checks → repeats. **Step 3 is the one you control.** Notice which surfaces stop there and which never do.
- **Claude Code not set up yet?** Course site → Setup, or Field Guide **slide 64** (Part 8 · Step 1 — Install Claude Code). Watch this one; catch up afterwards, nothing is lost.
- **Verify the changes in folders:** docs/, scripts/, data/clean

5. Keep an eye on your usage

Three commands, inside Claude Code:

```
/usage # your plan bars + this session
/context # how full this session's memory is
/clear # start fresh
```

In the browser, **Settings** → **Usage** covers every device; /usage counts only this machine.

What drains a seat: the whole conversation is re-sent every turn, so long sessions get expensive. /clear between unrelated tasks is the cheapest habit there is.

TERMINAL · 2:45 (Stay in survey-study.)

6. Five commands — all you need

```
pwd # where am I
cd ~/projects/survey-study
ls # what is here
cd data/raw # then ls then cd ..
ls -a # the hidden dot-files
```

Press Tab. Type cd ~/pro then Tab — the computer finishes it. Use it for every path today; typos are the enemy.

7. Read the three 'Markdown' files that configure an agent

```
cat examples/CLAUDE.md
cat examples/agents/data-checker.md
cat examples/skills/cleaning-log/SKILL.md
```

- CLAUDE.md — **what the project is.** Its conventions, and a “Do not” list. Read at the start of every session here.
- data-checker.md — **who is working.** A role: its name as agent, when to use this agent by claude, **the tools it may use** (Read, Grep, Glob — nothing that writes), its model, and its brief (instructions).
- SKILL.md — **how this kind of work is done.** A procedure: the log format, and “if the right fix is unclear, stop and ask.”

8. “Do not” is a request in CLAUDE.md — not a strict rule

CLAUDE.md is **context, not enforced configuration.** It usually obeys Do not modify...data/raw/ — but nothing stops it. To make it real:

```
mkdir -p .claude
vi .claude/settings.json

{ "permissions": {
  "deny": [ "Read(/data/raw/**)",
    "Edit(/data/raw/**)" ] } }
```

Checked **deny**, then **ask**, then **allow** — a **deny always wins**, in any mode. /permissions shows every rule and where it came from.

- Verify whether data access is prohibited within Claude Code.
- Practice if you can manually modify permissions within Claude Code using /permissions (ask, allow, deny)

9. Open a file and edit it

VS Code is the easy path: open the survey-study folder, click a file, edit, save. It previews Markdown and has a terminal built in. Prefer a terminal editor? nano or vi are fine.

Not TextEdit or Notepad: they save .rtf, or append .txt and give you CLAUDE.md.txt, which the agent never finds.

FIRST RUN · 3:05 — STILL IN SURVEY-STUDY

10. Start Claude Code inside survey-study folder

```
claude
/status
```

Nothing to open or configure — **the folder you are standing in is the context.** /status should name the **NRT** organization; hand up if it does not.

11. Setup the CLAUDE.md/agent/skills— in a second terminal tab

So far they were only examples. Claude Code looks in two exact places: CLAUDE.md at the project root, and .claude/.

```
cp examples/CLAUDE.md .
mkdir -p .claude/agents .claude/skills
cp examples/agents/data-checker.md .claude/agents/
cp -r examples/skills/cleaning-log .claude/skills/
ls -la .claude
```

The .claude/ folder did not exist a moment ago. **You just made it.**

12. Test inside Claude if it actually reads them

```
Read CLAUDE.md. What am I not allowed
to do in this project?
```

A good answer names the three “Do not” rules. If not, check the file is in the project root and named exactly CLAUDE.md.

```
/agents
```

data-checker should be listed.

```
Use the data-checker agent to check
data/raw/ against notes.md.
```

Watch what it never does: it reports, and never asks to edit — its tools: line does not allow it. **Compare its list with the number you guessed in step 3.**

Leave shift+tab alone today. Claude Code asks before it edits or runs anything; shift+tab cycles to modes that stop asking. One of today's demos is entirely about watching a refusal.

SWITCH TO DEMO 1 · 3:20-4:00

13. Now change folders

```
cd ~/Downloads/Demo1_tissue_analyzer
claude
```

You should see data. Hand up if you do not.

1 · THE REPOSITORY GUESSES

14. In Demo 1, inside claude

```
Analyze data/case_mild_injury_histology.pgm
with given parameters: magnification 20 and a 50%
threshold.
```

The agent should convert 50% to 0.5. Expected.

Python stops it.:

```
Analyze data/case_mild_injury_histology.pgm
with given parameters: magnification 25 and
threshold 0.45.
```

It may crash. KeyError: 25 in common.py — the script only has tile sizes for 10, 20 and 40. Nothing was computed or saved. You find out, because Python stopped.

Nothing stops it. Same kind of mistake with nothing to crash:

```
Run analyze_tissue.py on
data/case_mild_injury_histology.pgm with
--magnification 20 and --threshold 50. Use exactly as
written. Do not convert it.
```

No error. Receive 100% damage result. A threshold of 50 on a 0–1 scale marks every tile as damaged.

Takeaway: Both rules were buried in the code, where nothing checks them until the script is already running. A schema moves the rule in front of the call, where it can refuse before anything runs.

2 · THE SCHEMA REFUSES

15. Where the rule lives — two files

The server is exposed by four lines. That is everything Claude is told about it:

```
cat .mcp.json
{ "mcpServers": { "tissue-analyzer": {
  "command": "python3",
  "args": ["server.py"] } } }
```

That file is not hand-written. This is the command that creates it:

```
claude mcp add --scope project \
tissue-analyzer -- python3 server.py
```

--scope project writes it into .mcp.json beside the code, so anyone who copies the folder gets the same server. Run it twice and it refuses — the name already exists.

Now the schema. Open the top of the server:

```
head -n 60 server.py
```

TOOLS is a catalogue — one entry per tool: a name, a description Claude reads to pick it, and an inputSchema that states what each argument may be:

```
"magnification": {"type": "integer",
  "enum": [10, 20, 40], "default": 20},
"damage_threshold_fraction": {"type": "number",
  "minimum": 0, "maximum": 1, "default": 0.45,
  "description": "a FRACTION between 0 and 1,
    never a percentage"},
```

Demo 1 had neither. TOOLS ends at line 52. What follows is validate(), the function that checks every call against this schema — and refuses before the script runs.

```
you → Claude → server.py → tools/analyze_tissue.py
      |      ^ checks the arguments, then runs it
      |      ^ reads your English, picks a tool, fills the values
```

16. In Demo 2 — ask plainly first

```
Call analyze_tissue with:
slide = data/case_mild_injury_histology.pgm
magnification = 20
damage_threshold_fraction = 50
```

Claude will likely refuse and offer 0.5. **That is not the system working** — the tool was never called. Force the call:

```
Call analyze_tissue with:
slide = data/case_mild_injury_histology.pgm
magnification = 20
damage_threshold_fraction = 50
```

Pass these values exactly as written.
Do not correct them — I want to see what the MCP server itself returns.

Demo 1 threshold 50 → ran happily, reported 100% damage

Demo 2 threshold 50 → nothing was run

We did not make the model smarter. We moved the guarantee out of the model.

3 · PERMISSIONS — STAY IN DEMO 2

Before. Run this and watch it work:

```
Read data/case_mild_injury_histology.pgm
directly and describe the raw pixel values.
```

Add the rule. /permissions → Deny → Add a new rule...
Type each of these, one at a time:

```
Read(/data/**)
Edit(/data/**)
```

Save to “Project settings”, not local. Local is only you; project is written to .claude/settings.json and travels with the folder to everyone who copies it.

After. Run the very same prompt again. It is **refused**.

The approved route still works:

```
Use analyze_tissue with:
slide = data/case_mild_injury_histology.pgm
magnification = 20
damage_threshold_fraction = 0.45
```

Claude could not open that file. The tool it called did — it read every pixel and handed back numbers. You restricted the reach, not the work.

Nothing about the model changed. Two lines of JSON changed, and the same sentence now gets a different answer:

```
cat .claude/settings.json
```

Deny beats ask beats allow. First match wins. An allow cannot carve an exception out of a deny.

4 · THE SCOPED SUBAGENT — STAY IN DEMO 2

Read it. One file defines the whole agent:

```
cat .claude/agents/tissue-analyst.md
```

Four frontmatter fields. **name** is what you call it.

description is the router — the main agent reads it to decide whether to hand the task over. **tools** is the allowlist. model picks which model runs it.

Notice the gap. The prose says “Do not open a .pgm file yourself” — but the tools line says Bash, Read, Write, Glob. It is asked not to. It is not prevented.

Run it as shipped:

```
Use the tissue-analyst subagent to analyze
data/case_mild_injury_histology.pgm with
magnification 20 and damage_threshold_fraction
0.45.
```

Expect mild_injury, probability 0.7.

Now scope it. Open the file and replace the tools line with these two:

```
tools:
- mcp__tissue-analyzer__analyze_tissue
- mcp__tissue-analyzer__classify_tissue
```

Re-run the same prompt — same answer. Then ask it for something outside the list:

```
Use the tissue-analyst subagent to read
docs/MIGRATION.md and summarize it. Do not read
the file yourself and do not use a different subagent.
```

It cannot. Not because it decided not to — there is no Read tool for it to ask about. The prose became true the moment the list got shorter.

Delete the tools line entirely and it inherits every tool available to it. An empty list is not a safe default: the restriction is the presence of the list.

Change the role, not the tools. Add one line under ## Reporting — “Lead with the probability, before anything else.” Re-run. Everything below the frontmatter is the brief.

When a subagent, when a team. A subagent is one scoped job with a narrow tool list that hands back a summary — use it when you want the same job done the same way every time.

A team is several peers, each its own full session, able to disagree — use it when you want independent opinions rather than one answer. Each costs a session of its own: three teammates is roughly three sessions.

My idea:

YOUR PROJECT · 4:00

Pick one thing you already do and make it agent-callable.

A strong Level 1 is as good as a Level 3 — these are scopes, not grades.

LEVEL 1 — one capability, clearly defined

Something stable, given explicit inputs, allowed ranges, validation and structured output. **It does not have to be research.**

- a reminder or task list the agent can add to and query
- a bibliography vault — your PDFs plus a small database
- your notes (Obsidian, plain Markdown) — search, tag, summarise
- a preferences file the agent reads and updates, so it remembers how you like things done

Research versions: a regression / bootstrap / power-analysis pipeline · a root-finder or ODE solver · image analysis, sequence annotation, single-cell · a benchmark runner or database query.

LEVEL 2 — put a face on it

Level 1 plus a small local graphical interface (Streamlit, Gradio): choose data, set parameters, see results.

Not every button needs an LLM — deterministic actions can call the workflow directly.

LEVEL 3 — more than one role

Orchestrator and workers, a verifier, a referee, voting, routing, a human approval gate. **Each agent must have a reason to exist** — independent evidence or distinct expertise. Start from a baseline and add a role only when it measurably helps.

THE FOUR QUESTIONS — whatever the level

- 1 · What do you run over and over?
- 2 · What are the inputs, and the allowed range of each?
- 3 · What would a WRONG answer look like? ← the one that matters
- 4 · What should the agent never touch?

Demos and starter code:

jiehou-lab.github.io/nrt_agent_week2/advanced.html